

Homework 2: Language Models and Author Identification

Rey Stone

September 21, 2026

1 Instructions

In this assignment, you will implement a BPE-based tokenizer, several N-gram language models, and a perplexity measure. You will then use these components to implement a simple language modeling approach to author identification. Based on some training materials, you will be asked to use your LM to classify a set of text segments according to their author.

2 Normalization

Before starting the tokenization process, I have made several decisions to influence the tokenization process that are noteworthy.

- **Preserve case and punctuation** Because both Tolkien and Doyle write in a similar adventure narrative style, surface stylistic habits (such as capitalization, and punctuation patterns) might be useful in distinguishing between the different authors. This might add vocabulary sparsity, but *it's a sacrifice I'm willing to make*.
- **Stripped Gutenberg front-matter at the start and end of the .txt files** For both books, title blocks, and everything before “Chapter 1” was stripped because there is no added benefit in keeping these sentences in the text as they are not the author’s written work and would just clog up the vocabulary.
- **Got rid of Chapter indicators and Chapter titles** Chapter headers “Chapter XX” and the chapter titles are not inherently indicative of authorial style, and had to be removed.
- **Get rid of some fancy non-ASCII characters and replace them with their Unicode “equivalent”** Because punctuation was kept, there were some instances of non-ASCII characters still scattered throughout. Curly quotes were standardized to straight quotes since these artifacts are not authorial choices. Em-dashes were left untouched because collapsing them to a hyphen would make them indistinguishable from hyphenated compounds that may or may not exist in both texts.

In some instances in *The Lost World* there were instances of double hyphens or quadruple hyphens for em-dashes which had to be normalized to either single or double emdashes.

In four cases in *The Hobbit* the phrase “Gandalf’s” was missing an apostrophe for the possessive, which was normalized.

2.1 Difficulties

With normalization, I didn't want to make any hasty generalizations, so writing regex expressions from scratch took me like 4 hours that I did not need to spend time on.

3 Passages and splits

3.1 Passages

I went for a more nuanced approach to preserve as much of the author's style as possible. For this reason when creating train, dev, and test splits, I made the choice to split passages of the book instead of just pure sentences.

- Split each book into paragraphs on blank lines (`\n \n+`) rather than every new line since the source-text is hard-wrapped meaning new lines would appear in the middle of sentences or mid-paragraph and not just between paragraphs).

Result: 2001 paragraphs from *The Hobbit*, 1192 paragraphs from *The Lost World*

- Grouped consecutive paragraphs into passages, closing a passage once it reaches a word count of 80 (although in practice most ended up being 120 to 150 words long).

Result: 801 passages from *The Hobbit*, 497 from *The Lost World*

- Any leftover passages at the end of the books were simply merged into the second to last passage rather than discarded.

3.2 Train/dev/test splits

- Rejected fully random passage shuffling because neighboring passages from the same scene could end up split across the train/dev/test splits which would inflate apparent performance.
- Instead: grouped passages into contiguous blocks of size 10, shuffled the blocks with a fixed random seed of 1337 (yes, I know it's basic but I don't care), and assigned whole blocks to train/dev/test set at roughly 80/10/10.
- Each book (author) was split independently so train/dev/test each contain passages from both books/authors.
- Design rationale: Block-level shuffling helps with two things: 1. avoiding any leaks from adjacent/related text landing in different splits, and 2. it still samples across the whole book.

3.3 Difficulties

Some main difficulties I encountered with the design was deciding how to split the training data. Obviously just choosing sentences at random was never in my original plan, yet splitting paragraphs also seems counterintuitive since randomizing paragraphs across books isn't the best way to split up things either. Initially I was considering splitting up passages randomly, but having them be consecutive passages would preserve author style even more which is crucial in author identification, so I settled on that.

4 Tokenizer

- Followed Hugging Face base implementation in the quicktour section (complete with a BPE model `Whitespace` pre-tokenizer, and `BpeTrainer`), per assignment instructions (I really hope I did that correctly) to NOT implement BPE from scratch and not use pre-trained tokenizers by Hugging Face.
- Trained the tokenizer using only the **training split** (combined across both authors).
- Used `train_from_iterator` rather than `train` so training could operate directly on the in-memory passages instead of raw files.
- Wrapped the tokenizer in a function with vocabulary size and a `pre_tokenizer` as parameters to support looping over hyper-parameters for exploration.
- Both books n-gram models share the same tokenizer/vocabulary so their probability outputs are computed over the same symbol set making them comparable.
- `vocab_size` for smoothing comes from `tokenizer.get_vocab_size()`, not a count derived from the corpus, so out-of-training-tokens are still accounted for.

4.1 Padding and boundary marking

- Added `<s>` and `</s>` markers to mark the beginning and end of a sentence as special tokens alongside `[UNK]` (which was implemented in the quick-tour), so passages could be padded for n-gram boundary handling. `vocab_size` for smoothing comes from `tokenizer.get_vocab_size()`, not a count derived from the corpus, so out-of-training-tokens are still accounted for.
- Applied padding identically before training and scoring to avoid training/scoring mismatch.

4.2 Difficulties

Some of my main frustrations came from conflating “how many times a token appears” with “how many times a token is followed by something”. This was caught with hand-written checks of toy sentences by iterating over my code and writing it by hand. It was fixed by storing a separate counter for context totals separate of unigram counts.

5 N-gram model design

- One N-Gram model style class handles both bigram and trigram via an `ngram` parameter, rather than making separate functions for both.
- Add-k smoothing: $P(w|context) = (count(context, w) + k / count(context) + k \cdot V)$ was added with parameter `k` so experimenting with different `k` values could be tested.
- Verified correctness by hand-deriving expected probabilities on small corpuses and checking that the implementation matched, plus systematic sum-to-1 checks across `k=1` and `k=0.1` to confirm denominator scaling `k` was correct.
- Negative log probability of a passage: $sum(-log(P(...)))$ over all scored positions with bigram starting at position 2 since there are double `<s>` for trigram models (trigram implements this the same way). Perplexity was computed by summing the negative log probability and token counts across *all* passages in a set before diving and exp-ing rather than averaging per-passage

perplexity. This helped longer passages contribute proportionally more, which helps when passage lengths vary.

6 Perplexity experiments

Small grid over $ngram \in \{2, 3\}, k \in \{1, 0.1, 0.01\}$, each model evaluated on it's own author's dev set.

Results:

book	k	ngram	perplexity
hobbit	1	2	189.6
lost	1	2	214.7
hobbit	1	3	553.0
lost	1	3	571.9
hobbit	0.1	2	101.7
lost	0.1	2	115.2
hobbit	0.1	3	289.1
lost	0.1	3	298.8
hobbit	0.01	2	103.1
lost	0.01	2	118.5
hobbit	0.01	3	212.0
lost	0.01	3	211.1

6.1 Findings

The bigram model consistently scored better than the trigram model at every k , contrary to my initial hypothesis which was “more context would help”.

Explanation: with a vocabulary of ~ 100 and a corpus of only one book per author, the trigram space is too sparse. Because trigrams are more unique, they may not have been seen in training, so add-k smoothing would push that to be more uniform rather than a learned one. With the bigrams, since they would appear more in the corpus, the model could still make meaningful estimates.

Perplexity dropped sharply from $k = 1$ to $k = 0.1$ which makes sense because the add-k smoothing (according to Professor Martin) “makes the zeroes go away!”, and thus results in a more uniform distribution. When the k is smaller, the “real” training distribution is more represented with a smidge of “make the zeroes go away”. However, with $k = 0.01$ and bigram model, the perplexity rose slightly since the micro k penalizes unseen bigrams a bit too harshly. $k=0.1$ is the best middle ground between over-smoothing and under-smoothing.

6.2 Vocabulary size

This was added afterwards because I forgot. But I tested for the difference in vocabulary size with the following results for the perplexity:

vocab	hobbit-own(n=2, k=0.1)	lost-own(n=2, k=0.1)	hobbit-on-lost/lost-on-hobbit
500	63.9	70.1	129.3/125.3
1000	102.3	111.9	292.5/282.6
2000	199.4	218.4	668.8/644.3

The findings are as follows:

- **Absoulte perplexity scales with vocab size at every k and every n.** This makes sense since a bigger vocabulary means more possible bigrams and trigrams to spread the same traning data across so the model estimates are less confident. The smaller the vocab, the more repeats per context, which leads to lower perplexity.
- A large vocabulary makes trigrams sparser than the bigram space so the trigram degrades faster as the vocabulary size gets bigger.

One more important thing to note: Even though the models did better with the vocab size of 500, that just means it has fewer options to choose from, making the prediction problem easier for it.

6.3 Author discrimination checks

Gotta discriminate somewhere (please do not mark me off points for including this or cancel me).

Each trained model was scored on both it's own books and on each others:

model on	hobbit dev	lost dev
hobbit_model	~ 180-190	~ 280-290
lost_model	~ 280-290	~ 180-200

Each model performed worse on the other author's text, confirming the models capture distinct styles before testing on the test set.

7 Author Identification

First, final settings for my model:

I will be testing the author identification sets with a bigram model and $k=0.1$ with a vocab size of 1000 as my final parameters of choice based on the current data I have available for me.

Classifies a passage by comparing the `neg_log_prob` under both trained models for the same passage and picking the lower one.

Since we have not gotten the actual dataset yet the function `predict_author` was created to test this in one go.